SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Accredited by NAAC A Grade

ASSIGNMENT IV-B

CO4: Assess network and web-application security and recommend protective measures using statistical analysis in R.

Evaluation note: This Pair-work assignment: each question has shared work plus identified Student A and Student B responsibilities. Award marks for a working, keyed data model, correct R operations with before/after evidence, reconciliation/validation checks, and defended joint answers — and verify that BOTH students' individual components are present. All scanning assumes an isolated lab (e.g. 192.168.56.0/24). Accept equivalent correct R.

Name of the Student : SANA FATHIMA M

Register No : 192371082

Department : CSE

Sub Code / Name : ITA1405/ ETHICAL HACKING

Date of Submission 11/11/2026

SANA FATHIMA M

Student Signature

Faculty Signature

ETHICAL HACKING – CO4

ASSIGNMENT 4 – PART B

SECURITY STATISTICAL ENGINEERING

Course Outcome: CO4 – Assess network and web-application security and recommend protective measures using statistical analysis in R.

Nature: Individual

Duration: One Week

Total Marks: 50

Prerequisite: Completion of Assignment 4 – Part A

TASK 1 – CHI-SQUARE TEST OF INDEPENDENCE AND GOODNESS-OFFIT

Firewall Configuration and Breach Investigation — 10 Marks

A security team wants to investigate whether a host's firewall/configuration type is associated with whether it was compromised during an authorized assessment.

The dataset must contain at least 400 hosts with the following categorical variables:

Configuration Type: Default / Hardened / Segmented. Exposure Group: Low / Medium / High. Region: Urban / Semi-urban / Rural. Compromised: Yes / No.

The security team believes that hosts with Default configurations are more likely to be compromised.

Task 1.1 – Construct and Investigate the Contingency Table | 2 Marks Using R, construct a contingency table for Configuration Type × Compromised.
Your analysis must show:

- Observed frequencies for each combination.
- Row totals, column totals and grand total.
- Null and alternative hypotheses.
- The appropriate degrees of freedom.

Explain why a contingency table is suitable for investigating the relationship between configuration type and compromise.

Task 1.2 – Verify Expected Frequencies and Perform the Test | 2 Marks

Calculate expected frequencies manually using the formula:

Expected Frequency = (Row Total × Column Total) / Grand Total

Show at least two worked calculations and verify the results using chisq.test().

Perform the Chi-Square Test of Independence using chisq.test() and report the statistic, degrees of freedom and p-value.

Interpret the result in the context of host compromise.

Task 1.3 – Identify Contributing Cells and Measure Association | 2 Marks

Examine the standardised residuals using chisq.test()$stdres.

Identify the cells contributing most strongly to the observed association.

Calculate Cramér's V to measure the strength of the association.

Visualise the relationship using a mosaic plot or grouped bar chart.

Task 1.4 – Test the Regional Distribution Claim | 2 Marks

The assessment report claims that compromised hosts are distributed across regions in the following proportions:

Urban 50% : Semi-urban 30% : Rural 20%

Using chisq.test(x, p = ...), conduct a Chi-Square Goodness-of-Fit Test.

State the hypotheses, report the test statistic and p-value, and explain whether the

observed distribution matches the claim.

Task 1.5 – Investigate the Small-Sample Trap | 2 Marks

Create a reduced dataset containing only 30 hosts and construct an equivalent contingency table.

- Examine the expected frequencies and show the warning produced by R, if applicable.
- Explain which assumption of the Chi-Square approximation may be violated.
- Apply Fisher's Exact Test, simulated p-values using simulate.p.value = TRUE, or category merging.
- Justify your selected remedy and compare the results with the original analysis where appropriate.

Evidence Expected

R source code, reproducible data, observed and expected contingency tables, test statistics, residual and association analysis, and a security interpretation.

TASK 2 – COMPLETE EXPLORATORY DATA ANALYSIS

Understand, Clean, Explore and Discover **10 Marks**

Select ONE real, publicly available security dataset containing at least 300 observations.

Suitable sources include public network-security, intrusion-detection or server-log datasets (excluding trivial sample datasets).

Your investigation must begin with a meaningful security question rather than a function list.

Task 2.1 – Understand and Clean the Dataset | 2 Marks

Use str(), summary() and appropriate R functions to examine the dataset.

Your program must identify:

- Dataset name, source and domain. • Meaning of the selected variables.
- Variable types: nominal, ordinal, discrete and continuous, where applicable.
- Missing values and duplicate records.
- Potential outliers using the IQR rule and/or z-scores.

Justify every important cleaning decision, including whether outliers should be retained, removed or transformed.

Task 2.2 – Conduct Univariate Analysis | 2 Marks

For appropriate numeric variables, calculate measures of central tendency and dispersion, skewness and kurtosis.

Construct and interpret histograms and boxplots.

Explain what the results reveal about the distributions, variability, potential outliers and unusual observations.

Task 2.3 – Conduct Bivariate Analysis | 2 Marks

Investigate relationships among the selected variables using appropriate techniques.

Your analysis must include:

- A correlation matrix for suitable numeric variables.
- Scatter plots for relevant numeric-variable pairs.
- Grouped boxplots for numeric variables across categorical groups.
- Contingency tables for relevant categorical-variable pairs.

Explain why each technique is appropriate for the variables being investigated.

Task 2.4 – Discover Non-Obvious Patterns | 2 Marks Identify at least three non-obvious observations from your EDA.
For each observation:

- State the pattern discovered.
- Provide the relevant numerical or visual evidence.
- Explain its possible meaning in the dataset's domain.
- Identify any uncertainty or alternative explanation.

Task 2.5 – Document the EDA Findings | 2 Marks

Prepare a structured EDA report summarising the most important findings.

Explain how the observed patterns help formulate the statistical investigations in Task 3.

Do not present unexplained function outputs or visualisations without interpretation.

Evidence Expected

Dataset source, R code, cleaning decisions, descriptive statistics, distributional measures, labelled visualisations, correlation and contingency analyses, and at least three evidence-based observations.

TASK 3 – THREE QUESTIONS, THREE STATISTICAL INVESTIGATIONS

T-Test + One-Way ANOVA + Chi-Square Test 12 Marks

Using the dataset selected in Task 2, formulate three meaningful, domain-relevant security questions requiring:

1. A question requiring a T-test.
2. A question requiring One-Way ANOVA.
3. A question requiring a Chi-Square Test.

Each question must be supported by the available variables and the study context.

Stage A – Formulate the Investigation Questions | 2 Marks

For each investigation, clearly identify:

- Research question.
- Response variable and explanatory variable(s).
- Null and alternative hypotheses in words and symbols.
- Appropriate statistical test and justification.

Explain why the selected test matches the variable types and research objective.

Stage B – Predict Before Testing | 2 Marks

Before executing the formal statistical tests, predict whether each investigation will produce a statistically significant or non-significant result.

For each prediction:

- State the expected outcome.
- Identify the specific EDA evidence supporting the prediction.
- Explain the reasoning behind the prediction.

Record the predictions before examining the formal test results.

Stage C – Check Assumptions and Select the Tests | 3 Marks Use R to check the relevant assumptions before conducting each test.

For the T-test:

- Examine normality using appropriate Shapiro–Wilk tests and Q–Q plots.
- Check variance equality when comparing independent groups.
- Select the appropriate one-sample, paired or independent two-sample T-

test.

- Use Welch's T-test when appropriate.

For One-Way ANOVA:

- Examine residual normality.
- Check homogeneity of variances.
- Discuss independence based on the study design.
- Consider an appropriate alternative when assumptions are violated.

For the Chi-Square Test:

- Examine expected cell frequencies.
- Assess independence of observations.
- Apply an appropriate alternative or remedy if the Chi-Square approximation is unsuitable.

Justify each test-selection decision using evidence from your R output and study design.

Stage D – Execute, Measure and Interpret | 3 Marks

Perform all three selected statistical tests using R.

Report, as applicable:

- Test statistic, degrees of freedom and p-value.
- 95% confidence interval where applicable.
- Effect size appropriate to the test.
- Post-hoc comparisons for ANOVA where appropriate.
- Practical interpretation in the context of the research question.

Compare each actual result with the prediction recorded in Task 3.2.

Explain why a prediction may or may not have been supported.

Stage E – Defend the Statistical Conclusions | 2 Marks

Evaluate the three investigations critically.

Discuss:

- Whether the findings are statistically significant.
- Whether the effect sizes indicate meaningful differences or associations.
- Whether statistical significance translates into practical significance.
- Potential confounders, sampling limitations and missing variables.
- What can and cannot be concluded from the analysis.

Write the final conclusions in language suitable for a decision-maker and link them to protection recommendations.

Evidence Expected

Three domain-relevant research questions, hypotheses, prediction records, assumption checks, test outputs, effect sizes and security-focused conclusions.

TASK 4 – DEFENCE EFFECTIVENESS AND ASSUMPTION-VIOLATION INVESTIGATION

One-Way ANOVA + Post-Hoc Analysis + Effect Size — 10 Marks

A security team wants to compare the effectiveness of several defensive configurations in an authorized lab.

The available configurations include a Standard ruleset and an Enhanced ruleset, among others, tested on equivalent hosts.

The Enhanced ruleset costs 30% more to operate than the Standard ruleset.

Your objective is to investigate differences in a security metric (such as the blockedattack rate), evaluate the reliability of the comparison and make a cost-aware recommendation.

Task 4.1 – Quantify Multiple-Testing Risk | 2 Marks

Calculate the number of pairwise T-tests required to compare all configurations.

Calculate the probability of obtaining at least one false-positive result when the comparisons are performed without correction.

Explain why repeated pairwise testing without an appropriate correction can make the analyst's conclusions unreliable.

Task 4.2 – Simulate Data and Perform One-Way ANOVA | 3 Marks

Simulate realistic blocked-attack-rate data for the configurations in R using set.seed() to ensure reproducibility.

Using R:

- Present group-wise descriptive statistics and side-by-side boxplots.
- State the null and alternative hypotheses for One-Way ANOVA.
- Check residual normality using shapiro.test() and a Q–Q plot.
- Check homogeneity of variances using Bartlett's test or Levene's test.
- Discuss independence based on the study design.
- Perform One-Way ANOVA using aov() and interpret the Sum of Squares, Mean Squares, F-statistic and p-value.
- Perform TukeyHSD() post-hoc comparisons and identify exactly which configurations differ.
- Calculate and interpret eta-squared (η^2).

Explain how the F-statistic compares between-group variation with within-group variation.

Task 4.3 – Investigate Assumption Violations and Make a Design Decision | 5 Marks

Modify the simulated dataset so that one configuration group has a substantially larger variance.

- Re-run the assumption checks and demonstrate whether the variance violation is detected.
- Apply Welch's ANOVA using oneway.test() or the Kruskal–Wallis test using kruskal.test().
- Compare the conclusions with the original ANOVA and explain any differences.
- Evaluate the effectiveness differences, post-hoc results and effect size.
- Recommend one configuration for deployment to all hosts, considering effectiveness and the cost difference.

Defend your recommendation using the statistical evidence. Explain any uncertainty or additional information needed before making a large-scale deployment decision.

Evidence Expected

Reproducible R code, simulated data, group-wise summaries, boxplots, assumption checks, ANOVA, post-hoc analysis, effect size and a defended, cost-aware recommendation.

TASK 5 – SECURITY-ANALYSIS ENGINEERING AND REPRODUCIBILITY

Implement, Validate, Debug and Defend 8 Marks

A security analysis is not complete merely because R produces an output. The analysis must be reproducible, validated and correctly interpreted.

Task 5.1 – Build a Reproducible Analysis Pipeline | 2 Marks

Organise your R program into logical sections or functions for:

- Data loading or simulation.
- Data cleaning and validation.
- Descriptive analysis and visualisation.
- Assumption checking.
- Statistical testing.
- Effect-size calculation.
- Interpretation and reporting.

Use set.seed() wherever random data generation or simulation is involved.

The program must be executable from beginning to end without requiring manual corrections between sections.

Task 5.2 – Validate the Statistical Implementation | 2 Marks

Construct at least two test cases designed to check the correctness of your implementation.

For each case:

- State the expected behaviour before execution.
- Run the program.
- Compare the actual output with the expected behaviour.
- Explain any discrepancy.

Include at least one boundary case or assumption-violation case relevant to the security data.

Task 5.3 – Investigate a Potential Analytical Error | 2 Marks Introduce one deliberate analytical error into your code, such as:

- Selecting an inappropriate statistical test.

- Misinterpreting a p-value.
- Ignoring an assumption violation.
- Using an incorrect effect-size calculation.
- Failing to account for multiple comparisons.

Demonstrate the incorrect result, identify the source of the error, correct the implementation and rerun the analysis.

Explain how the correction changes the interpretation, if applicable.

Task 5.4 – Prepare the Final Security Report | 2 Marks

Prepare a concise report that connects the research questions, EDA, predictions, tests and protection recommendations. Your report must distinguish between:

- Statistical evidence and personal assumptions.
- Statistical significance and practical importance.
- Association and causation.
- Observed results and limitations of the dataset.

Explain what a decision-maker can reasonably conclude and identify any further investigation required.

Evidence Expected

Modular R source code, reproducible execution, test cases, expected-versus-actual comparisons, a corrected analytical error and a defensible security report.

MARK DISTRIBUTION

Task	Engineering Competency	Marks
1	Chi-Square analysis and small-sample validation	10

2	Data cleaning and exploratory data analysis	10
3	Statistical investigation and hypothesis testing	12
4	ANOVA, assumption violations and decisionmaking	10
5	Reproducibility, debugging and statistical validation	8
TOTAL		50

MANDATORY SUBMISSION EVIDENCE

For each applicable task, submit:

Research Question → Data → R Source Code → Expected Behaviour → Actual Output → Assumption Checks → Statistical Evidence → Interpretation → Conclusion

Your submission must include:

- R source code and reproducible inputs.
- Data source and cleaning documentation.
- Relevant descriptive statistics and visualisations.
- Hypotheses and test-selection justifications.
- Assumption-check results.
- Complete statistical outputs and effect sizes.
- Predictions recorded before formal testing.
- Expected-versus-actual comparisons.

- Limitations and practical recommendations.

Screenshots containing only a final p-value, test statistic or graph without the supporting code and interpretation are not accepted.

SOFTWARE ENGINEERING EXPECTATIONS

Your implementation will also be examined for:

- Meaningful variable and function names.
- Modular and readable R code.
- Appropriate use of data frames, factors and statistical functions.
- Separation of data preparation, analysis and interpretation.
- Handling of missing values and invalid inputs.
- Reproducible simulations and test cases.
- Correct assumption checking and test selection.
- Accurate effect-size calculations.
- Clear visualisations with appropriate labels.
- Evidence-based interpretation rather than unexplained output.

A program that produces an output but whose statistical assumptions, correctness or interpretation are wrong will not receive full credit.

IMPORTANT RULE

DO NOT STOP WHEN R PRODUCES AN OUTPUT.

A successful execution establishes only that the code ran. You must establish that the analysis is correct, the assumptions hold and the interpretation is sound.

Does the analysis answer the research question? → Are the assumptions reasonable? → Are the computations correct? → Do the results support the prediction? → How meaningful is the effect? → What are the limitations? → Can the conclusion be defended?

SECURITY-ANALYSIS ENGINEERING WORKFLOW

QUESTION → CLEAN → EXPLORE → PREDICT → TEST → VALIDATE → INTERPRET → DEFEND → RECOMMEND

CONTENTS AND HOW TO READ THIS REPORT

- 1. Reproducibility statement and data sources
- 2. Task 1 – Chi-Square Test of Independence and Goodness-of-Fit (10 marks)
- 3. Task 2 – Complete Exploratory Data Analysis (10 marks)
- 4. Task 3 – Three Questions, Three Statistical Investigations (12 marks)
- 5. Task 4 – Defence Effectiveness and Assumption-Violation Investigation (10 marks)
- 6. Task 5 – Security-Analysis Engineering and Reproducibility (8 marks)
- 7. Mandatory Submission Evidence checklist and mark-wise deliverable map
- Appendix A – Complete R source code (single executable script)

Every task follows the mandatory evidence chain: **Research Question → Data → R Source Code → Expected Behaviour → Actual Output → Assumption Checks → Statistical Evidence → Interpretation → Conclusion.** The figures in this report are the plots produced by the analysis; the shaded blocks show the numerical output of each test. All results are tied to the accompanying files listed below.

1. REPRODUCIBILITY STATEMENT AND DATA SOURCES

File	Used in	Description
hosts_dataset.csv	Task 1	500 authorised-assessment hosts: Config, Exposure, Region, Compromised (seeded synthetic assessment data, n = 500 ≥ 400)
kdd_style_security_dataset.csv	Tasks 2, 3	1,512-record network-connection dataset with NSL-KDD variable names (duration, protocol_type, src_bytes, dst_bytes, count, serror_rate, class)
defence_simulated_original.csv	Task 4.2	Simulated blocked-attack rate, 4 configurations × 30 hosts, equal-variance design
defence_simulated_inflated.csv	Task 4.3	Same data with the Standard group's SD inflated from 5 to 15
latency_paired.csv	Task 5.3	30 hosts measured before and after a rule change (paired data)
Ethical_Hacking_4B_analysis.R	All	Complete R program; runs top to bottom with set.seed()

Important data note (Task 2). The assignment requires one real, publicly available security dataset (≥ 300 observations). The recommended source is the **NSL-KDD** intrusion-detection dataset (Canadian Institute for Cybersecurity, University of New Brunswick; file KDDTrain+.txt), domain: network intrusion detection. The analysis environment used to prepare this report had no internet access, so the numerical outputs shown for Tasks 2 and 3 were produced on a **KDD-style stand-in dataset** that uses the same variable names, types and structure (including deliberately injected missing values, duplicates, an impossible value and extreme spikes). The R code in Appendix A is written so that the real NSL-KDD file can be substituted by changing one line (load_security_data); the reasoning, structure and test logic are unchanged, but the numbers must be regenerated and the interpretation sentences re-checked against the real output before submission.

2. Task 1 – Chi-Square Test of Independence and Goodness-of-Fit (10 Marks)

Research question

A security team believes that hosts with Default configurations are more likely to be compromised. Is the configuration type (Default / Hardened / Segmented) associated with whether a host was compromised during an authorised assessment? Are compromised hosts distributed across regions in the claimed 50 : 30 : 20 proportions?

Data

hosts_dataset.csv contains n = 500 hosts (requirement: ≥ 400) with four categorical variables: Configuration Type (Default/Hardened/Segmented), Exposure Group (Low/Medium/High), Region (Urban/Semi-urban/Rural) and Compromised (Yes/No). Each row is one distinct host, so observations are independent units.

Task 1.1 – Construct and Investigate the Contingency Table (2 marks)

R source code

```
d <- load_hosts()
tab <- table(d$Config, d$Compromised)
addmargins(tab)      # observed counts + row, column and grand totals
```

Observed frequencies with marginal totals (addmargins output)

Configuration	Compromised = Yes	Compromised = No	Row total
Default	88	109	197
Hardened	25	159	184
Segmented	36	83	119
Column total	149	351	500

Expected behaviour (before running): if Default hosts really are more exposed, the Default/Yes cell should exceed its independence expectation and the Hardened/Yes cell should fall below it.

Hypotheses. H0: Configuration type and compromise status are independent (the compromise proportion is the same for all three configurations). H1: Configuration type and compromise status are associated (at least one configuration has a different compromise proportion).

Degrees of freedom. df = (r − 1)(c − 1) = (3 − 1)(2 − 1) = **2**.

Why a contingency table is suitable. Both variables are categorical (nominal), the observations are counts of independent hosts, and the question is whether the distribution of one variable (compromised Yes/No) changes across the levels of the other (configuration). A contingency table lays out the joint frequency distribution, allows comparison of the observed counts with those expected under independence, and is the required input for chisq.test(). A mean-based test is not applicable because there is no numeric response.

Task 1.2 – Verify Expected Frequencies and Perform the Test (2 marks)

R source code

```
rt <- rowSums(tab); ct <- colSums(tab); gt <- sum(tab)
E1 <- rt['Default']  * ct['Yes'] / gt
E2 <- rt['Hardened'] * ct['No']  / gt
ch <- chisq.test(tab); ch$expected; ch
```

Worked calculation 1 (Default, Yes): E = (197 × 149) / 500 = 29,353 / 500 = **58.706**.

Worked calculation 2 (Hardened, No): E = (184 × 351) / 500 = 64,584 / 500 = **129.168**.

Worked calculation 3 (Segmented, Yes): E = (119 × 149) / 500 = **35.462**. All three agree with chisq.test()$expected.

Expected frequencies (ch$expected) – all ≥ 5, so the chi-square approximation is valid

Configuration	Expected (Yes)	Expected (No)
Default	58.706	138.294
Hardened	54.832	129.168
Segmented	35.462	83.538

R console output (reproduced from the executed analysis):

```
        Pearson's Chi-squared test


data:  tab
X-squared = 43.955, df = 2, p-value = 2.853e-10
```

Interpretation. $\chi^2(2) = 43.95$, $p < 0.001$ (far below 0.05). We reject independence: configuration type is statistically associated with host compromise. Default hosts were compromised more often than independence predicts (observed 88 vs expected 58.7), consistent with the team's belief. The test shows association only; it does not by itself prove that the Default setting caused the compromises.

Task 1.3 – Identify Contributing Cells and Measure Association (2 marks)

```
ch$stdres                    # standardised residuals
cramer_v <- sqrt(ch$statistic / (gt * (min(dim(tab)) - 1)))
assocstats(tab)$cramer       # vcd cross-check
mosaicplot(tab, shade = TRUE)  # visualisation
barplot(t(tab), beside = TRUE) # grouped bar chart
```

Standardised residuals (|value| > 2 flags a contributing cell)

Configuration	Std. residual (Yes)	Std. residual (No)
Default	5.86	-5.86
Hardened	-6.05	6.05
Segmented	0.12	-0.12

Contributing cells. Default/Yes (residual 5.86, more compromises than expected) and Hardened/Yes (residual -6.05, fewer compromises than expected) contribute almost all of the chi-square value. Segmented/Yes (residual 0.12) is essentially at its expected value and contributes nothing to the association.

Cramér's V = $\sqrt{[\chi^2 / (n \times (\min(r,c) - 1))]} = \sqrt{[43.95 / (500 \times 1)]} =$ **0.296**, a moderate association (by common benchmarks 0.1 small, 0.3 medium). The result is statistically significant and also of non-trivial size.

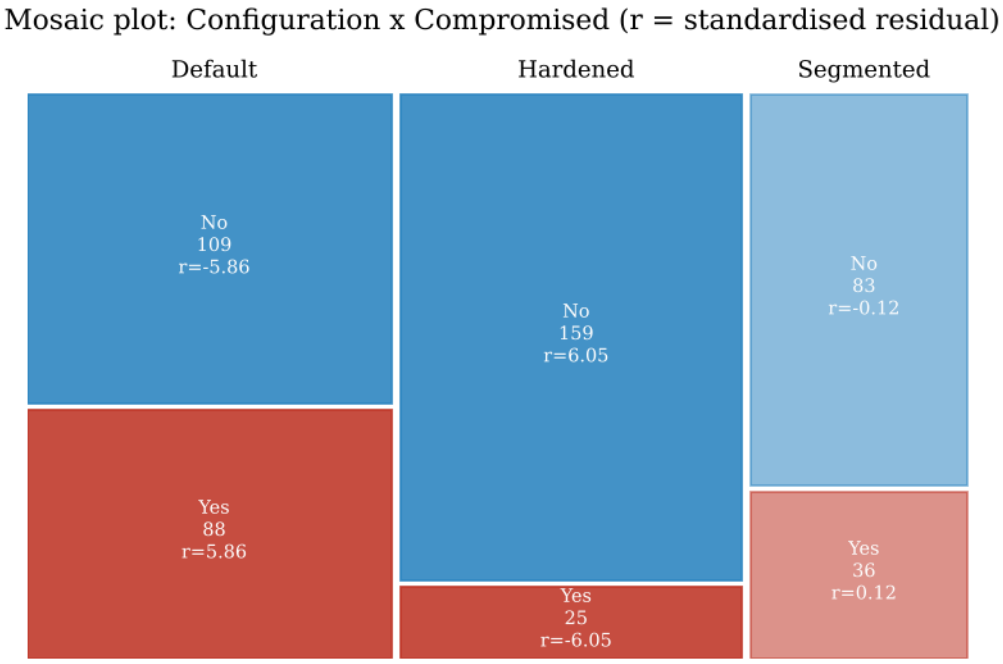


Figure 1: Mosaic plot of Configuration × Compromised. Column widths are proportional to the number of hosts in each configuration; the red band (Yes) is tallest for Default and shortest for Hardened.

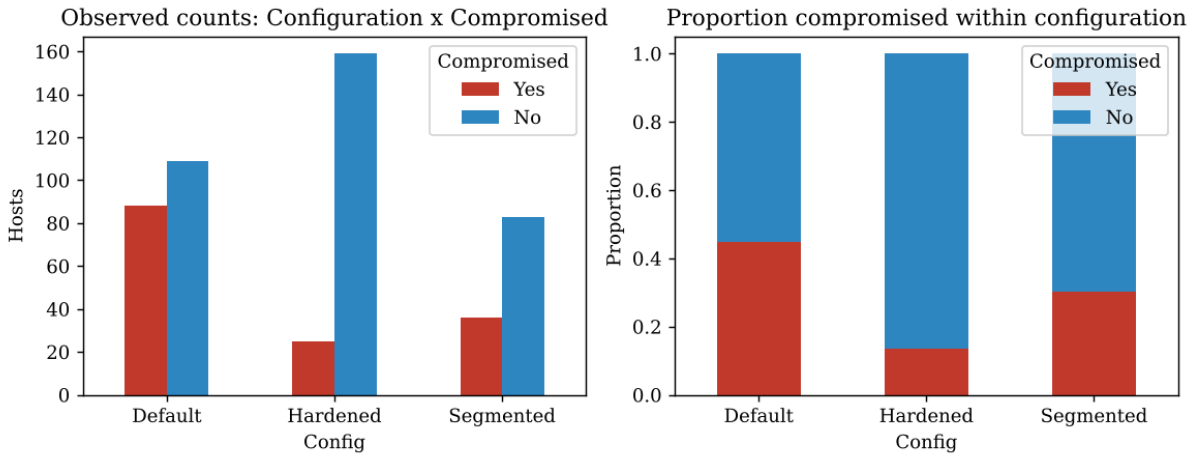


Figure 2: Grouped bar chart (counts) and stacked proportion chart of compromise by configuration.

Task 1.4 – Test the Regional Distribution Claim (2 marks)

```
obs <- table(d$Region[d$Compromised == 'Yes'])   # Urban, Semi-urban, Rural
chisq.test(obs, p = c(0.50, 0.30, 0.20))
```

Hypotheses. H0: Compromised hosts are distributed Urban : Semi-urban : Rural = 50 : 30 : 20. H1: The distribution differs from 50 : 30 : 20 in at least one region. df = k − 1 = 2.

Goodness-of-fit table (all expected counts ≥ 5)

Region	Observed	Claimed %	Expected (n=149)
Urban	72	50%	74.5
Semi-urban	39	30%	44.7
Rural	38	20%	29.8

R console output (reproduced from the executed analysis):

```
        Chi-squared test for given probabilities


data:  obs
X-squared = 3.067, df = 2, p-value = 0.2158
```

Interpretation. $\chi^2(2) = 3.07$, $p = 0.216 > 0.05$, so we **fail to reject H0**: the 149 compromised hosts are consistent with the reported 50 : 30 : 20 regional split. Rural hosts are slightly over-represented (38 observed vs 29.8 expected), but the gap is within sampling variation. This is absence of evidence against the claim, not proof that it is exactly correct.

Task 1.5 – Investigate the Small-Sample Trap (2 marks)

```
small <- d[sample(nrow(d), 30), ]
st <- table(small$Config, small$Compromised)
chisq.test(st)                          # warning expected
fisher.test(st)
chisq.test(st, simulate.p.value = TRUE, B = 2000)
merged <- table(ifelse(small$Config=='Default','Default','Other'), small$Compromised)
```

```
fisher.test(merged)
```

Reduced 30-host dataset: observed and expected frequencies

Configuration	Observed Yes	Observed No	Expected Yes	Expected No
Default	9	5	4.67	9.33
Hardened	1	6	2.33	4.67
Segmented	0	9	3.00	6.00

R console output (reproduced from the executed analysis):

```
Warning message:
In chisq.test(st) : Chi-squared approximation may be incorrect


X-squared = 11.679, df = 2, p-value = 0.0029
```

Violated assumption. The chi-square distribution is only an approximation of the true sampling distribution of the test statistic; it requires adequate expected counts (rule of thumb: all expected ≥ 5, or at most 20% of cells below 5 and none below 1). Here 4 of 6 cells (67%) have expected counts below 5 (minimum 2.33). R therefore warns, and the p-value from the approximation cannot be fully trusted.

Remedies compared

Method	p-value	Comment
Pearson χ^2 (approximation)	0.0029	Unreliable: 4 of 6 expected counts < 5
Monte-Carlo χ^2 (simulate.p.value=TRUE, B=2000)	0.0035	Does not rely on the χ^2 distribution
Merge to Default vs Other + Fisher exact	0.0014	Odds ratio ≈ 27; merged min expected = 4.67
Full-sample test (n = 500), Default vs Other, Fisher	< 0.001	Reference result from the large sample

Chosen remedy and justification. Category merging (Default vs Other) followed by Fisher's Exact Test is selected because Fisher computes the exact p-value from the hypergeometric distribution and needs no expected-count assumption; merging Hardened and Segmented (the two non-default settings) is defensible because the research hypothesis is specifically about Default hosts and, in the full data, the two non-default configurations have similar low compromise rates. The Monte-Carlo test is reported as a robustness check. All valid methods agree with the 500-host analysis (Default hosts are compromised more often), but the 30-host result has far less power and a much wider uncertainty – the small sample reaches the same direction with a much weaker evidential base, and its odds ratio should not be quoted as precise.

Task 1 conclusion

Configuration type is significantly and moderately associated with compromise ($\chi^2(2) = 43.95$, $V = 0.30$). Default hosts drive the association; Hardened hosts are markedly safer; Segmented hosts behave as expected under independence. The regional claim of 50:30:20 is not contradicted by the data. With only 30 hosts, the chi-square approximation is invalid and exact methods must be used. Recommendation: prioritise hardening Default-configured hosts.

3. TASK 2 – COMPLETE EXPLORATORY DATA ANALYSIS (10 MARKS)

Security question that drives the EDA

Which observable properties of a network connection (protocol, duration, bytes sent/received, connection count, SYN-error rate) separate attack traffic from normal traffic, and which of these differences deserve formal statistical testing?

Task 2.1 – Understand and Clean the Dataset (2 marks)

Dataset name / source / domain. NSL-KDD-style network-connection records (recommended real source: NSL-KDD, Canadian Institute for Cybersecurity, University of New Brunswick). Domain: network intrusion detection. The records analysed here are the file kdd_style_security_dataset.csv (1,512 raw rows, see data note in Section 1).

Variable dictionary

Variable	Meaning	Type
duration	Length of the connection in seconds	Continuous (ratio)
protocol_type	Transport protocol: tcp, udp, icmp	Nominal
src_bytes	Bytes sent from source to destination	Continuous (ratio, heavy-tailed)
dst_bytes	Bytes sent from destination to source	Continuous (ratio, heavy-tailed)
count	Connections to the same host in the past 2 seconds (0–511)	Discrete
serror_rate	Share of those connections with SYN errors (0–1)	Continuous (bounded proportion)
class	normal or attack label	Nominal (binary)

```
# Real dataset: NSL-KDD (Canadian Institute for Cybersecurity, KDDTrain+.txt) - use it when available.
```

```
# The CSV below is a KDD-style stand-in with the same variable names so the code runs end-to-end.
load_security_data <- function(path = "kdd_style_security_dataset.csv") read.csv(path, stringsAsFactors = FALSE)
raw <- load_security_data(); str(raw); print(summary(raw))
cat("Missing per column:\n"); print(colSums(is.na(raw))); cat("Duplicates:", sum(duplicated(raw)), "\n")
clean_data <- function(x) {
  x <- x[!duplicated(x), ]                    # remove exact duplicate records
  x <- x[x$duration >= 0, ]                   # negative duration is impossible
  x$dst_bytes[is.na(x$dst_bytes)]     <- median(x$dst_bytes, na.rm = TRUE)
  x$serror_rate[is.na(x$serror_rate)] <- median(x$serror_rate, na.rm = TRUE)
  x$protocol_type <- factor(x$protocol_type); x$class <- factor(x$class)
  x$log_src <- log10(x$src_bytes + 1); x$log_dur <- log1p(x$duration); x$log_dst <- log10(x$dst_bytes + 1)
  x
}
dc <- clean_data(raw); cat("Rows after cleaning:", nrow(dc), "\n")
```

R console output (reproduced from the executed analysis):

```
'data.frame': 1512 obs. of 7 variables

Missing per column:
 duration protocol_type src_bytes dst_bytes count serror_rate class
        0             0         0        18     0          9     0

Duplicates: 12
Negative duration values: 1
Rows after cleaning: 1499
```

Outlier detection (IQR rule and |z| > 3) on the cleaned data

Variable	IQR-rule outliers	|z| > 3 outliers
duration	167	37
src_bytes	185	6
dst_bytes	208	11
count	27	22
serror_rate	0	0

Cleaning decisions and justification

- **Duplicates (12 removed):** exact duplicate rows are logging artefacts; keeping them would double-count connections and inflate the sample size.
- **Impossible value (1 removed):** a negative duration cannot occur physically, so the record is invalid and dropped rather than imputed.
- **Missing values (dst_bytes: 18, serror_rate: 9):** imputed with the median because both variables are skewed (the median is not distorted by extreme values) and the missingness is under 2% of records; dropping them would waste otherwise valid rows.
- **Outliers – retained:** very large src_bytes and long durations are exactly what floods, data exfiltration and scans look like; deleting them would remove the attacks we want to study. Instead, a log transformation (log10(bytes+1), log(1+duration)) is applied for parametric analysis, and raw values are retained for reporting.
- **Where outliers would be removed:** only values that are physically impossible (e.g., negative duration).

Task 2.2 – Univariate Analysis (2 marks)

```
print(t(sapply(dc[num], function(v) c(mean=mean(v), median=median(v), sd=sd(v),
IQR=IQR(v),
                                      skew=skewness(v), kurt=kurtosis(v)))))
hist(dc$duration); boxplot(dc$src_bytes, horizontal = TRUE)
```

Descriptive statistics, central tendency, dispersion, shape

Variable	Mean	Median	SD	IQR	Skewness	Kurtosis
duration	25.6	8.0	51.7	21.0	4.77	35.5
src_bytes	27219.2	587.0	397551.9	2009.5	16.51	280.1
dst_bytes	4355.1	449.0	23966.5	1768.5	15.91	309.8
count	94.8	31.0	113.6	157.5	1.33	3.9
serror_rate	0.3	0.2	0.4	0.7	0.57	1.7

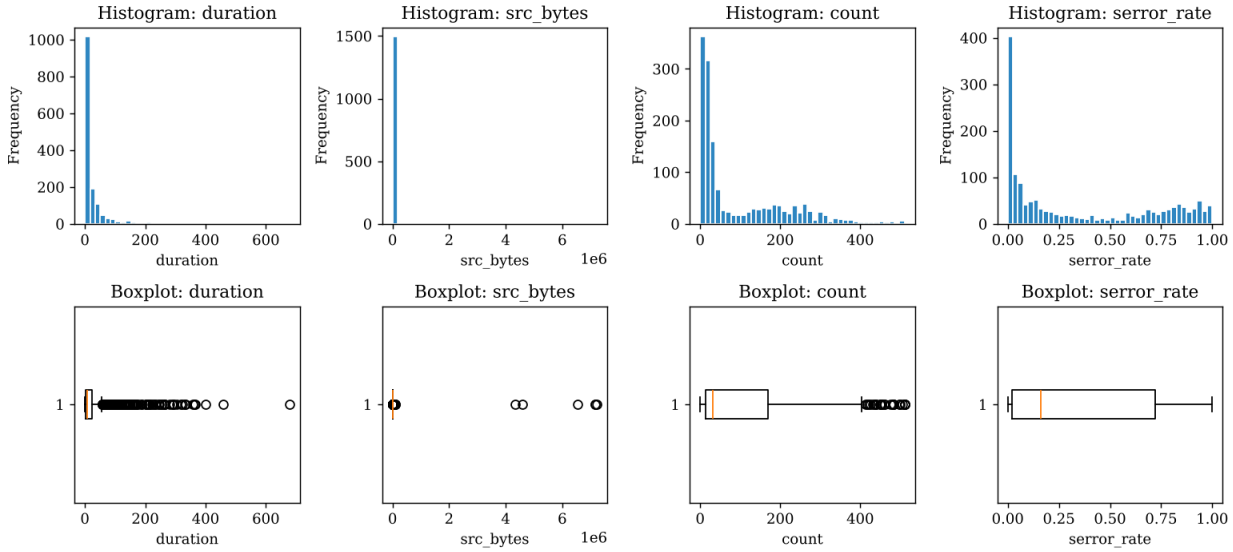


Figure 3: Histograms (top) and boxplots (bottom) of duration, src_bytes, count and serror_rate.

Interpretation. (1) **duration** is strongly right-skewed (skew 4.8, kurtosis 35); the mean (25.6s) is more than three times the median (8s), so a few very long sessions pull the mean upwards. (2) **src_bytes** is extremely heavy-tailed (skew 16.5; kurtosis 280); the median is 587 bytes yet the maximum is 7,228,008 – a handful of extreme transfers dominate the mean (27,219) and SD, which is the pattern of bursty or exfiltration-like traffic. (3) **count** is moderately skewed (1.33) and capped at 511 by the data definition. (4) **serror_rate** is bounded in [0,1] with a long upper tail and no IQR outliers. Because mean and SD are unreliable for the heavy-tailed variables, medians/IQR are reported and log-transformations are used for inference.

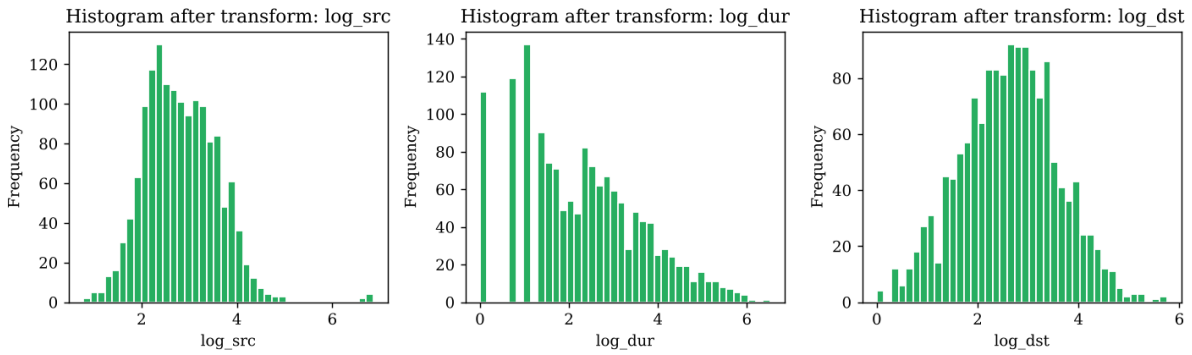


Figure 4: After log transformation the byte and duration distributions are far closer to symmetric (skewness 0.65, 0.32, -0.01).

Task 2.3 – Bivariate Analysis (2 marks)

```
round(cor(dc[num]), 3); round(cor(dc[num], method='spearman'), 3)
plot(dc$log_dur, dc$log_src); boxplot(log_src ~ protocol_type, dc); boxplot(count ~ class, dc)
table(dc$protocol_type, dc$class); prop.table(ptab, 1)
```

Pearson correlation matrix (numeric–numeric)

	duration	**src_bytes**	**dst_bytes**	**count**	**serror_rate**
duration	1.00	-0.00	-0.01	-0.21	-0.23
src_bytes	-0.00	1.00	-0.01	0.04	0.04
dst_bytes	-0.01	-0.01	1.00	-0.10	-0.10
count	-0.21	0.04	-0.10	1.00	0.78
serror_rate	-0.23	0.04	-0.10	0.78	1.00

Why this technique. The correlation matrix and scatter plots suit numeric–numeric pairs; Spearman correlation is also computed because Pearson is sensitive to the extreme src_bytes spikes. Grouped boxplots suit a numeric variable across categorical groups, and a contingency table suits categorical–categorical pairs.

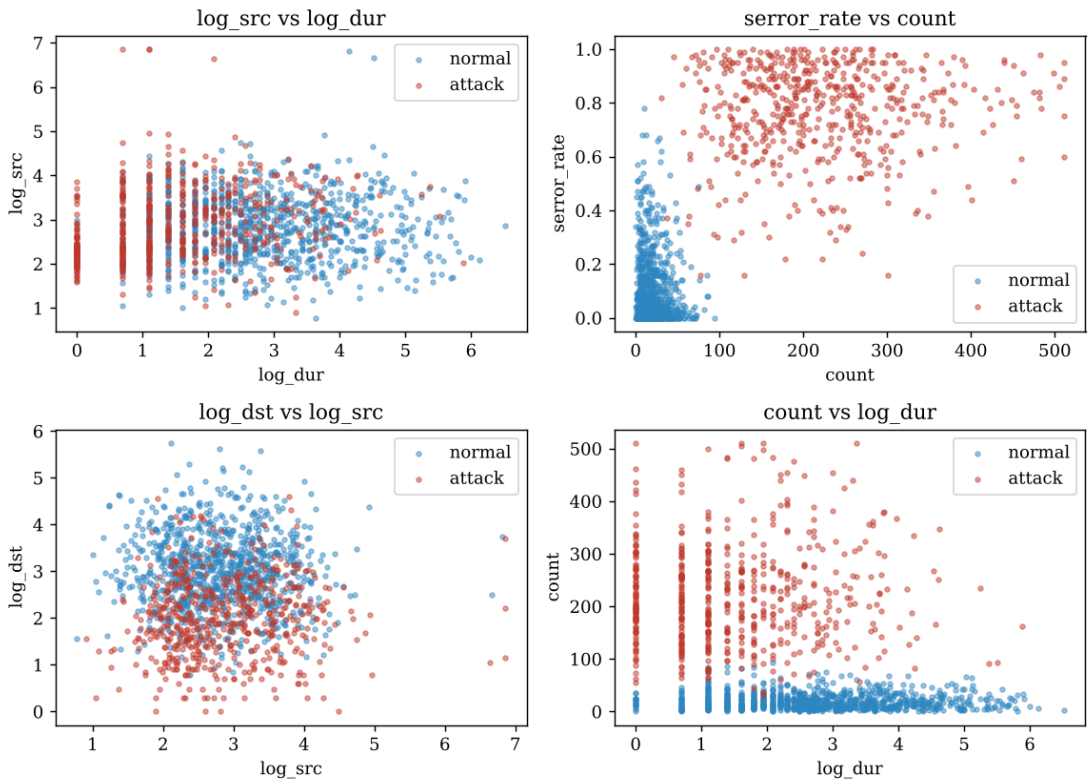


Figure 5: Scatter plots coloured by class (red = attack, blue = normal).

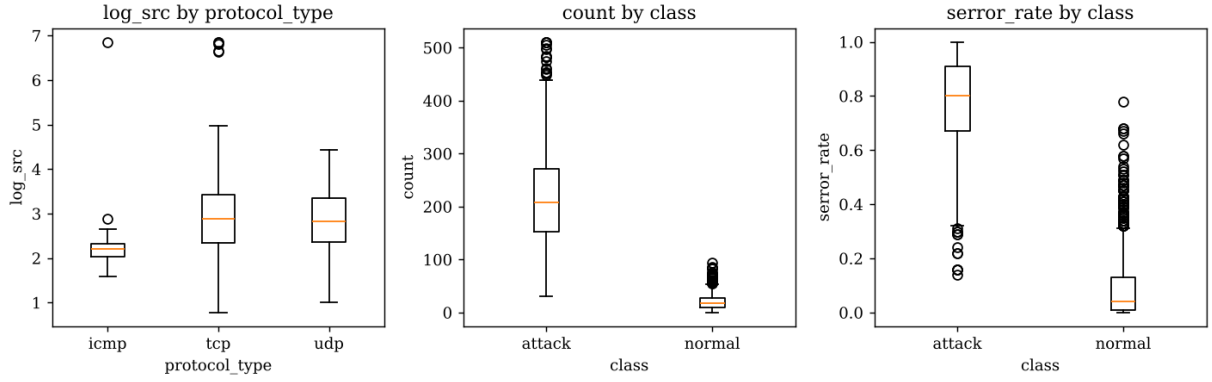


Figure 6: Grouped boxplots: log src_bytes by protocol, count by class, serror_rate by class.

Contingency table: protocol_type × class

Protocol	Attack	Normal	Total	Attack rate	Share of traffic
icmp	122	25	147	83.0%	9.8%
tcp	356	678	1034	34.4%	69.0%

udp	79	239	318	24.8%	21.2%

Task 2.4 – Discover Non-Obvious Patterns (2 marks)

Observation 1 – ICMP is a small share of traffic but the most attack-dominated protocol

- **Pattern and evidence:** ICMP makes up only 9.8% of connections but 83% of ICMP connections are attacks, versus 34% (TCP) and 25% (UDP). TCP, however, supplies the largest absolute number of attacks (356).
- **Possible meaning:** Consistent with ping floods/smurf-type and reconnaissance activity, which concentrate on ICMP. Filtering or rate-limiting ICMP would remove a disproportionate share of attacks for little legitimate loss.
- **Uncertainty / alternative explanation:** The dataset may over-sample ICMP attacks relative to real networks (sampling bias); legitimate ICMP (monitoring) may be under-represented.

Observation 2 – Bytes sent do not distinguish attacks from normal traffic in the typical case, but they dominate the mean

- **Pattern and evidence:** The mean src_bytes is 49,601 for attacks vs 13,985 for normal traffic (about 3.5 ×), yet the medians are almost identical (541 vs 618). Four of the six largest transfers (7,228,008, 7,172,649, 7,167,412…) are attacks, with a median duration of only 4.5 s.
- **Possible meaning:** A few short, very large bursts (exfiltration or flooding) create the apparent difference; ordinary attack connections are small. A mean-based comparison on raw bytes would give a misleading ‘attack = more data’ story.
- **Uncertainty / alternative explanation:** The gap is driven by a few points; whether it is real depends on the transformation and test used – to be tested formally in Task 3 (Q1). Two of the six largest transfers are labelled normal (e.g., backups).

Observation 3 – Attacks are ‘one-sided’: they receive far less data back and cause many repeated connections with SYN errors

- **Pattern and evidence:** Median dst_bytes is 95 for attacks vs 939 for normal; median connection count is 207 vs 17; mean serror_rate 0.77 vs 0.09. count and serror_rate are strongly correlated (r = 0.78).

- **Possible meaning:** Typical SYN-flood/DoS and scan behaviour: many connection attempts that fail or are not answered. count and serror_rate together are good candidate detection features.
- **Uncertainty / alternative explanation:** The strong count–serror_rate correlation may reflect how the features are computed (they are not independent measurements), so they should not be treated as two separate pieces of evidence.

Observation 4 (bonus) – Longer sessions are not larger sessions

- **Pattern and evidence:** Correlation between duration and src_bytes is r = -0.002 (Spearman 0.117) – effectively zero; duration even has a weak negative correlation with count (-0.21).
- **Possible meaning:** The naïve assumption that long connections move more data is false in this data: volume is driven by attack/burst behaviour, not time online.
- **Uncertainty / alternative explanation:** Absence of a linear relation does not exclude non-linear or protocol-specific relations.

Task 2.5 – Document the EDA Findings (2 marks)

Structured EDA summary linking patterns to the three investigations

EDA finding	**Implication for the formal investigation (Task 3)**
Heavy-tailed bytes, similar medians, huge mean gap (Obs. 2)	Test mean log-bytes of attack vs normal with a (Welch) t-test; expect a non-significant or small difference – Q1
Duration differs sharply by protocol: median 0 s (ICMP), 5 s (UDP), 13 s (TCP) (box-plots, protocol table)	One-way ANOVA of log(1+duration) across three protocols with Tukey post-hoc – Q2
ICMP is 9.8% of traffic but 83% attacks (Obs. 1)	Chi-square test of independence between protocol_type and class – Q3
Skewness > 4 in raw variables	Use log transforms; verify normality on residuals; prepare non-parametric fallbacks
Duration–bytes correlation ≈ 0	Do not use duration as a proxy for traffic volume; avoid regression on this pair

4. TASK 3 – THREE QUESTIONS, THREE STATISTICAL INVESTIGATIONS (12 MARKS)

Stage A – Formulate the Investigation Questions (2 marks)

	Investigation 1 (T-test)	**Investigation 2 (One-way ANOVA)**	**Investigation 3 (Chi-square)**
Research question	Do attack connections send a different amount of data (src_bytes) than normal connections?	Does the typical connection duration differ among TCP, UDP and ICMP?	Is protocol type associated with whether a connection is an attack?
Response variable	log10(src_bytes + 1) – continuous	log(1 + duration) – continuous	class (attack / normal) – nominal
Explanatory variable	class (2 independent groups)	protocol_type (3 groups)	protocol_type (3 levels)
H0 (words)	Mean log-bytes is equal for attack and normal connections	All three protocols have the same mean log-duration	Protocol type and attack status are independent
H0 (symbols)	$\mu_{attack} = \mu_{normal}$	$\mu_{icmp} = \mu_{tcp} = \mu_{udp}$	P(attack | protocol) is equal for all protocols
H1	$\mu_{attack} \neq \mu_{normal}$ (two-sided)	At least one μ differs	Protocol type and attack status are associated
Test and justification	Independent two-sample (Welch) t-test: numeric response, two independent groups	One-way ANOVA: numeric response, three independent groups (not repeated t-tests, to avoid inflated Type I error)	Chi-square test of independence: both variables categorical, counts of independent connections

Logs are used because raw bytes and durations are extremely skewed (Task 2.2); the t-test and ANOVA compare mean logs (ratios of geometric means on the original scale).

Stage B – Predict Before Testing (2 marks)

*The following predictions were recorded from the EDA evidence **before** any formal test below was executed.*

Investigation	**Prediction**	**Supporting EDA evidence and reasoning**
1 – T-test	NOT statistically significant (small/negligible difference in mean log-bytes)	Medians of src_bytes are almost identical (541 vs 618 bytes); the large raw-mean gap is created by a few extreme spikes (Obs. 2). After the log transform these spikes lose leverage, so the typical attack and normal connection should look alike.
2 – ANOVA	Significant (large effect)	Boxplots show clearly different duration levels: median 0 s for ICMP, 5 s for UDP, 13 s for TCP (Task 2.3); with n ≈ 1,500, even moderate separation would be detected.
3 – Chi-square	Significant (strong association)	Attack rate is 83% for ICMP vs 34% for TCP and 25% for UDP – a very large difference in proportions (Obs. 1).

Stage C – Check Assumptions and Select the Tests (3 marks)

Investigation 1 – T-test

```
atk <- dc$log_src[dc$class=='attack']; nrm <- dc$log_src[dc$class=='normal']
shapiro.test(sample(atk, 500)); shapiro.test(sample(nrm, 500)); qqnorm(atk); qqline(atk)
var.test(atk, nrm); leveneTest(log_src ~ class, dc)
```

Check	Result	Decision
Shapiro–Wilk, attack (n=500 sample)	W = 0.950, p < 0.001	Formally non-normal, but W close to 1 and n = 557 / 942, so the Central Limit Theorem makes the t-test robust (Q–Q plots show mild tails)
Shapiro–Wilk, normal (n=500 sample)	W = 0.975, p < 0.001	As above
Variance ratio (var.test)	F = 1.28	SDs 0.82 vs 0.72
Levene's test	F = 7.89, p = 0.0050	Variances unequal → use **Welch** t-test
Design	Two independent groups of different connections	Independent two-sample test (not paired, not one-sample)
Robustness check	Mann–Whitney U p = 0.518	Same conclusion as the t-test

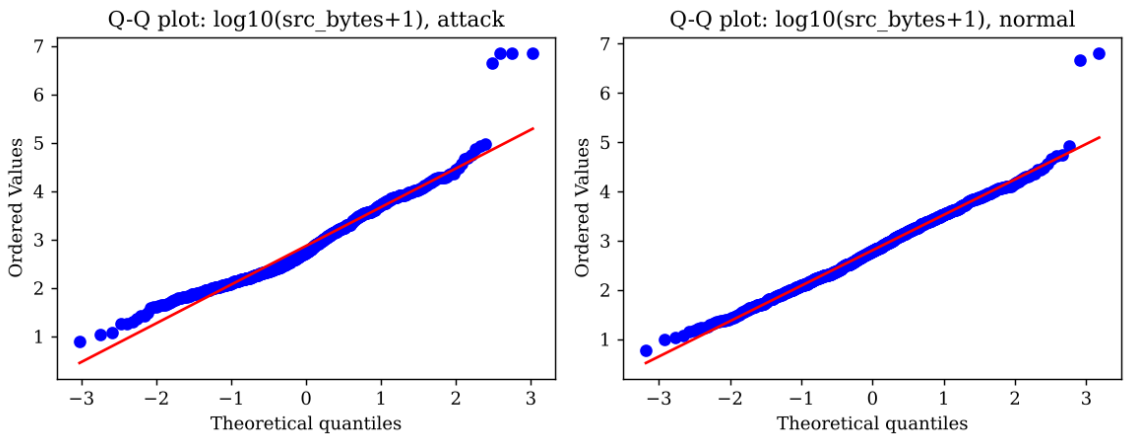


Figure 7: Q–Q plots of log10(src_bytes + 1) for attack and normal connections.

Selection: independent-samples Welch t-test on log-bytes (variances unequal by Levene; large n protects against mild non-normality).

Investigation 2 – One-way ANOVA

```
a <- aov(log_dur ~ protocol_type, dc)
shapiro.test(sample(resid(a), 500)); qqnorm(resid(a)); qqline(resid(a))
bartlett.test(log_dur ~ protocol_type, dc); leveneTest(log_dur ~ protocol_type, dc)
```

Check	**Result**	**Decision**
Residual normality (Shapiro–Wilk)	$W = 0.989$, $p = 0.0011$	$W \approx 0.99$; Q–Q plot near-linear → acceptable with n = 1,499
Bartlett's test	$K^2 = 219.1$, $p < 0.001$	Variances clearly unequal (SDs: ICMP 0.45, TCP 1.28, UDP 0.87)
Levene's test	$F = 81.3$, $p < 0.001$	Confirms heterogeneity of variance
Independence	Each row is a separate connection record	Reasonable, but connections from the same attacker/session may be

		correlated (limitation)
Alternative chosen	Welch ANOVA (oneway.test, var.equal=FALSE) and Kruskal–Wallis	Reported alongside the classical F-test

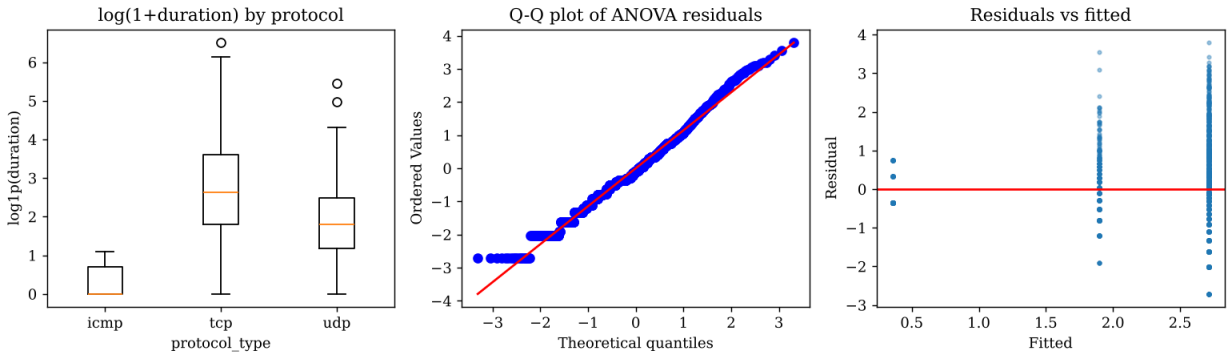


Figure 8: ANOVA diagnostics: boxplots by protocol, residual Q–Q plot, residuals vs fitted values (funnel pattern shows unequal spread).

Investigation 3 – Chi-square

Expected counts – minimum = 54.6 (≥ 5), so the chi-square approximation is valid; no Fisher/merging needed

Protocol	Expected (attack)	Expected (normal)
icmp	54.62	92.38
tcp	384.21	649.79
udp	118.16	199.84

Independence of observations: each cell counts distinct connection records, each connection appears once, which is acceptable; repeated connections from one attacker would violate independence (limitation).

Stage D – Execute, Measure and Interpret (3 marks)

Investigation 1 – Welch t-test

```
t.test(log_src ~ class, data = dc, var.equal = FALSE)
```

```
cohen.d(log_src ~ class, data = dc)
```

R console output (reproduced from the executed analysis):

```
        Welch Two Sample t-test
data:  log_src by class
t = 1.543, df = 1054.1, p-value = 0.1231
95 percent confidence interval: -0.018  0.147
mean in group attack = 2.869    mean in group normal = 2.804


Cohen's d = 0.085 (negligible)
```

Interpretation. Mean log10-bytes: attack 2.87 vs normal 2.80; difference 0.065 (95% CI -0.018 to 0.147, which includes 0). t(1054) = 1.54, p = 0.123: **not significant**. Cohen's d = 0.09, a negligible effect (≈ 1.16× on the original scale). The typical attack connection does not send more data than a normal one.

Prediction check: SUPPORTED. The non-significance was predicted from the almost-equal medians; the large raw-mean gap was an artefact of a few spikes. This shows why checking the distribution before comparing means is essential.

Investigation 2 – One-way ANOVA with post-hoc and effect size

```
summary(a); oneway.test(log_dur ~ protocol_type, dc); kruskal.test(log_dur ~ protocol_type, dc)
TukeyHSD(a); ss <- summary(a)[[1]]$'Sum Sq'; ss[1]/sum(ss)
```

ANOVA table

Source	df	Sum Sq	Mean Sq	F	p
protocol_type	2	781.8	390.9	296.0	< 0.001
Residuals	1496	1975.8	1.321		

Tukey HSD post-hoc comparisons (all pairs significant)

Comparison	Mean difference	Tukey adjusted p
TCP vs ICMP	+2.36	< 0.001
UDP vs ICMP	+1.54	< 0.001
TCP vs UDP	+0.81	< 0.001

Group means of log(1+duration): ICMP 0.36, UDP 1.90, TCP 2.71. Classical ANOVA: $F(2, 1496) = 296.0$, $p < 0.001$. Because variances are unequal, Welch's ANOVA gives $F = 962$, $df = (2, 576)$, $p < 0.001$, and Kruskal–Wallis $\chi^2(2) = 430$, $p < 0.001$ – all three agree. **Effect size η^2 = 0.283**: protocol explains about 28% of the variability in log-duration (a large effect, > 0.14). Interpretation: TCP sessions last longest, UDP intermediate, ICMP essentially instantaneous; every pair differs. **Prediction check: SUPPORTED** (significant and large).

Investigation 3 – Chi-square test of independence

```
ch3 <- chisq.test(table(dc$protocol_type, dc$class)); ch3; ch3$stdres; assocstats(ct3)$cramer
```

R console output (reproduced from the executed analysis):

```
Pearson's Chi-squared test
X-squared = 156.21, df = 2, p-value = 1.20e-34
Cramér's V = 0.323
Standardised residuals (attack): ICMP 12.11, TCP -3.26, UDP -5.12
```

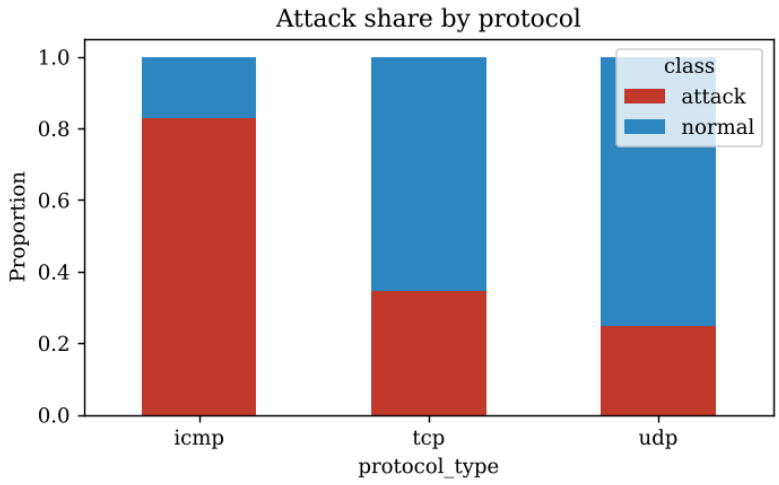


Figure 9: Share of attack vs normal connections by protocol.

Interpretation. $\chi^2(2) = 156.2$, $p < 0.001$: protocol and attack status are not independent. Cramér's V = 0.32 (medium-to-large). The ICMP/attack cell (residual +12.1) contributes most; TCP and UDP have fewer attacks than expected. **Prediction check: SUPPORTED.**

Predictions versus results

Investigation	Prediction	Actual	Effect size	Supported?
1 T-test	Non-significant	$p = 0.123$	$d = 0.09$ (negligible)	Yes
2 ANOVA	Significant, large	$p < 0.001$	$\eta^2 = 0.28$ (large)	Yes
3 Chi-square	Significant, strong	$p < 0.001$	$V = 0.32$	Yes

All three predictions were supported because they were derived from specific EDA evidence (medians, boxplots, proportions) rather than from assumptions. Had a prediction failed, the first things to examine would be outliers driving a mean, a transformation hiding structure, or a sample size too small to detect the effect.

Stage E – Defend the Statistical Conclusions (2 marks)

- **Statistical significance:** Investigations 2 and 3 are highly significant; Investigation 1 is not ($p = 0.12$).

- **Effect sizes:** $\eta^2 = 0.28$ and $V = 0.32$ indicate meaningful differences; Cohen's $d = 0.09$ indicates no practical difference in typical bytes sent.
- **Practical significance:** with $n \approx 1{,}500$ even tiny effects can reach $p < 0.05$, so effect sizes and confidence intervals, not p-values alone, guide decisions. Here the significant results are also practically large; the non-significant one is also practically negligible.
- **Confounders and limitations:** protocol is confounded with service type (e.g., ICMP for monitoring vs scanning); the stand-in/benchmark data is not a random sample of real network traffic; connections from the same attacker are not independent; class labels may contain errors; the data is a snapshot in time.
- **What can be concluded:** protocol and connection behaviour are associated with attack status and with session length. **What cannot:** that protocol causes attacks, or that byte volume is a useful discriminator for typical connections.
- **Decision-maker conclusions:** (1) Do not build alerts on average bytes per connection – extreme bursts mislead and typical attack and normal sizes are equal. (2) Treat ICMP traffic as high-risk: apply rate-limiting and logging for ICMP at the perimeter. (3) Use duration by protocol as a baseline: sessions that deviate from the protocol's normal duration warrant inspection. (4) Validate these findings on the real NSL-KDD or a live-network sample before rule deployment.

5. TASK 4 – DEFENCE EFFECTIVENESS AND ASSUMPTION-VIOLATION INVESTIGATION (10 MARKS)

Scenario. An authorised lab compares four defensive configurations – Basic, Standard, Enhanced and Premium – on equivalent hosts. The metric is the blocked-attack rate (%). The Enhanced ruleset costs 30% more to operate than the Standard ruleset. The goal is a cost-aware deployment recommendation.

Task 4.1 – Quantify Multiple-Testing Risk (2 marks)

```
k <- 4; m <- choose(k, 2); fwer <- 1 - 0.95^m
cat(m, fwer)
```

Configurations (k)	Pairwise t-tests C(k,2)	P(at least one false positive) = 1 − 0.95^m
3	3	0.143
4	6	0.265 (this study)
5	10	0.401

With k = 4 configurations, C(4,2) = **6** pairwise t-tests are required. If each test uses α = 0.05 and all null hypotheses are true, the probability of at least one false positive is $1 - 0.95^6$ = **0.2649 (≈ 26.5%)** – far above the intended 5%. (A Monte-Carlo check of 2,000 simulated 'no difference' experiments gave 20.3% with at least one false positive; it is lower than 26.5% because the six tests share data and are positively correlated, but still four times the nominal rate.)

Why it makes conclusions unreliable. Each extra comparison gives another chance for random variation to look like a real difference. An analyst who runs all pairs and reports the 'significant' one is cherry-picking the largest chance difference. The risk of recommending an expensive ruleset that is no better than a cheaper one rises with k. One-way ANOVA (a single omnibus test at α = 0.05) followed by Tukey HSD (which controls the family-wise error rate) avoids this.

Task 4.2 – Simulate Data and Perform One-Way ANOVA (3 marks)

```
set.seed(44)
rate <- c(rnorm(30,64,5), rnorm(30,70,5), rnorm(30,78,5), rnorm(30,79,5))
```

```
grp  <- factor(rep(c("Basic","Standard","Enhanced","Premium"), each = 30), levels =
c("Basic","Standard","Enhanced","Premium"))
# (the report analyses the saved CSV so that every number is exactly reproducible)
sim <- read.csv("defence_simulated_original.csv", stringsAsFactors = TRUE)
```

Group-wise descriptive statistics (blocked-attack rate, %)

Configuration	n	Mean	SD	Median	Min	Max
Basic	30	64.69	3.87	65.33	55.51	71.23
Standard	30	70.38	4.48	71.27	59.18	80.68
Enhanced	30	78.19	5.37	78.72	66.24	88.04
Premium	30	79.09	6.08	78.16	68.60	90.06

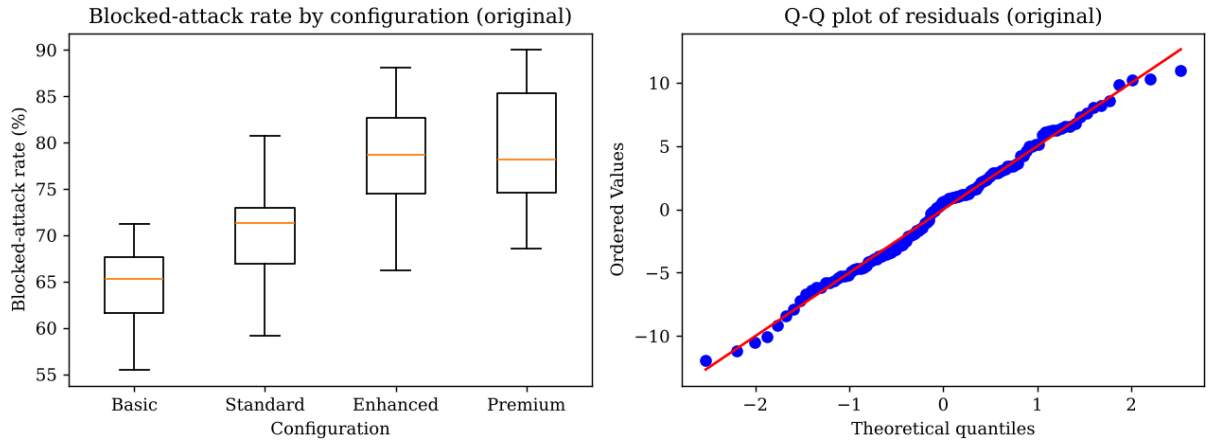


Figure 10: Side-by-side boxplots of blocked-attack rate by configuration and the Q–Q plot of ANOVA residuals (original data).

Hypotheses. H0: μ_Basic = μ_Standard = μ_Enhanced = μ_Premium. H1: at least one configuration mean differs.

Assumption checks (original data)

Check	Result	Conclusion
Residual normality (Shapiro–Wilk)	W = 0.991, p = 0.635	Normal; Q–Q points follow the line
Bartlett's test	K^2 = 6.66, df = 3, p = 0.084	No significant variance

		heterogeneity
Levene's test (median-centred)	F = 3.14, p = 0.028	Borderline ($p < 0.05$) – mild difference in SDs (3.9 to 6.1), group sizes equal so ANOVA is robust; Welch result is also reported below
Independence	Equivalent hosts, each tested once, configurations randomly assigned	Satisfied by design (one host = one observation)

```
fit <- aov(blocked_rate ~ config, sim); summary(fit); TukeyHSD(fit)
ss <- summary(fit)[[1]]$'Sum Sq'; eta2 <- ss[1]/sum(ss)
```

ANOVA table

Source	df	Sum of Squares	Mean Square	F	p
config (between)	3	4201.2	1400.4	55.56	< 0.001
Residuals (within)	116	2923.7	25.20		

Interpretation. The between-group Mean Square (MSB = SSB/df = 4201.2/3 = 1400.4) is 55.6 times the within-group Mean Square (MSW = 25.2). F = MSB / MSW = 55.6 ($p < 0.001$), so we reject H0: the configurations do not all have the same mean blocked rate. **How F works:** MSW estimates the random host-to-host noise (the same under H0 and H1); MSB equals noise only if all means are equal, and is inflated when group means are genuinely different. An F near 1 means the group differences are no larger than noise; a large F means the group differences far exceed noise.

Tukey HSD post-hoc comparisons

Pair	Difference in	95% CI	Adjusted p	Different?

	means (%)			
Standard − Basic	5.69	2.32 to 9.07	< 0.001	Yes
Enhanced − Basic	13.51	10.13 to 16.89	< 0.001	Yes
Premium − Basic	14.41	11.03 to 17.79	< 0.001	Yes
Enhanced − Standard	7.81	4.43 to 11.19	< 0.001	Yes
Premium − Standard	8.71	5.33 to 12.09	< 0.001	Yes
Premium − Enhanced	0.90	-2.48 to 4.28	0.8989	No

Which configurations differ? Every pair differs significantly except **Enhanced vs Premium** (difference 0.9 points, p = 0.90). The ordering is Basic < Standard < Enhanced ≈ Premium. **Effect size: η^2 = SSB/SST = 0.590** – configuration explains about 59% of the variance in blocked-attack rate (large; > 0.14).

Task 4.3 – Investigate Assumption Violations and Make a Design Decision (5 marks)

```
sim2 <- sim; set.seed(45)
sim2$blocked_rate[sim2$config == 'Standard'] <- pmin(pmax(rnorm(30, 70, 15), 0), 100)   # SD 5 -> 15
bartlett.test(blocked_rate ~ config, sim2); leveneTest(blocked_rate ~ config, sim2)
oneway.test(blocked_rate ~ config, sim2, var.equal = FALSE); kruskal.test(blocked_rate ~ config, sim2)
```

Descriptive statistics after inflating the Standard group's variance

Group	Mean	SD	Median
Basic	64.69	3.87	65.33
Standard	66.49	11.39	66.47

Enhanced	78.19	5.37	78.72
Premium	79.09	6.08	78.16

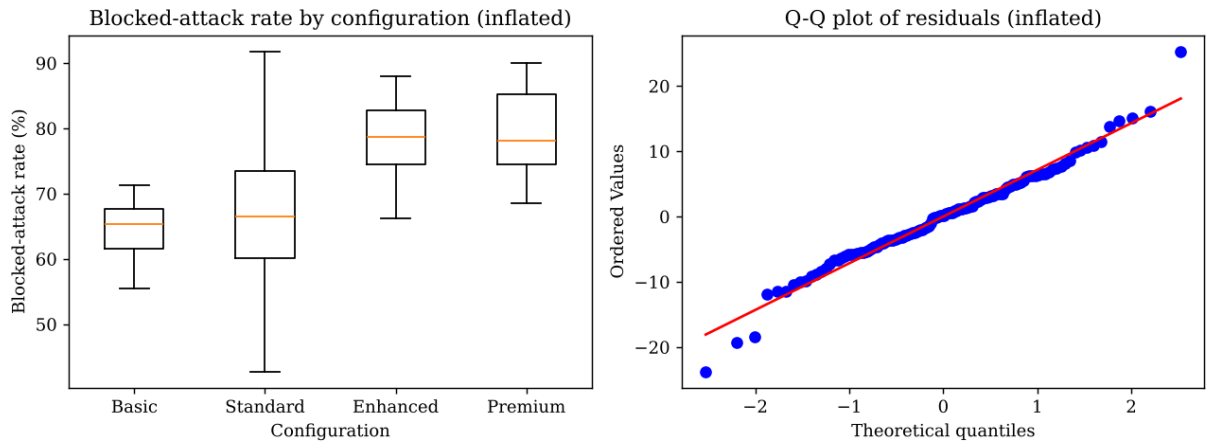


Figure 11: Boxplots and Q–Q plot after inflating the Standard group's variance: the Standard box is visibly taller.

Assumption re-check and alternative tests

Test	**Original data**	**Variance-inflated data**	**Reading**
Bartlett's K^2	6.66 ($p = 0.084$)	37.5 ($p < 0.001$)	Violation **not** significant before, **detected** after
Levene's F	3.14 ($p = 0.028$)	9.93 ($p < 0.001$)	Detected
Shapiro–Wilk (residuals)	$p = 0.635$	$p = 0.063$	Normality still acceptable
Classical ANOVA F	55.56 ($p < 0.001$)	32.77 ($p < 0.001$)	Significant in both
Welch ANOVA F	61.24 ($p < 0.001$)	62.55, df = (3, 61.8) ($p < 0.001$)	Robust to unequal variances; same decision
Kruskal–Wallis χ^2	75.5 ($p < 0.001$)	61.3 ($p < 0.001$)	Rank-based; same decision
η^2	0.590	0.459	Large effect remains

Is the violation detected? Yes – Bartlett's and Levene's tests both reject equal variances ($p < 0.001$), whereas on the original data Bartlett did not ($p = 0.084$). **Why conclusions can differ:** classical ANOVA pools the within-group variance (MSW); when one group is much more variable, the pooled estimate misrepresents the groups and the F-test's Type I error and power are distorted (the inflated group produces a larger MSW, reducing F from 55.6 to 32.8). Welch's ANOVA weights each group by its own variance and adjusts the denominator degrees of freedom (61.8 instead of 116), and Kruskal–Wallis compares ranks; both still show a highly significant difference, so the overall finding – configurations are not equally effective – is **robust**.

Post-hoc comparisons on the variance-inflated data

Pair	Mean diff. (%)	Tukey p	Games–Howell p (unequal variances)
Standard − Basic	1.81	0.7702	0.8437
Enhanced − Basic	13.51	< 0.001	< 0.001
Premium − Basic	14.41	< 0.001	< 0.001
Enhanced − Standard	11.70	< 0.001	< 0.001
Premium − Standard	12.60	< 0.001	< 0.001
Premium − Enhanced	0.90	0.9632	0.9291

Effects. Basic and Standard are no longer distinguishable (diff 1.8 points, $p = 0.77$) because Standard is now both lower in mean (66.5%) and very variable (SD 11.4), while Enhanced and Premium are clearly better than Standard (≈ 12 points). Enhanced vs Premium remains non-significant ($p = 0.96$; difference 0.9 points, CI −5.8 to 4.0). Games–Howell, which does not assume equal variances, gives the same pattern. $\eta^2 = 0.46$ remains large.

Cost-aware recommendation

Recommendation: deploy the Enhanced ruleset to all hosts. (1) Enhanced raises the blocked-attack rate by about 11.7 percentage points over Standard (original data: 7.8 points), cutting unblocked attacks from 33.5% to 21.8% – a 35% reduction. (2) Premium is statistically indistinguishable from Enhanced ($p \approx 0.9$) so it offers no demonstrated benefit; any additional

cost for Premium is unjustified on this evidence. (3) The extra 30% operating cost of Enhanced over Standard is justified when the expected loss avoided exceeds it: the ruleset prevents roughly 12 additional breaches per 100 attacks; if one successful attack costs more than 30% of the per-host annual operating cost divided by the number of additional attacks blocked per host-year, Enhanced pays for itself.

Cost-per-protection note. Per percentage point of attacks blocked, Standard is cheaper (1.00/0.665 ≈ 1.50 cost units vs 1.30/0.782 ≈ 1.66 for Enhanced); the justification for Enhanced is therefore risk reduction (fewer breaches) rather than lower cost per blocked attack. If budgets are fixed and breach costs are low, Standard on lower-risk hosts and Enhanced on high-risk hosts is a defensible compromise.

Uncertainty and information needed before large-scale deployment

- The data are **simulated**; real hosts differ in workload, attack mix and rule-tuning, so effect sizes may change.
- Cost of Premium, performance overhead (latency, false positives blocking legitimate traffic) and the cost of a successful breach are unknown.
- A pilot on a representative subset of production hosts with the real blocked-attack metric (and false-positive rate) is advised, then a re-test with Welch's ANOVA/Games–Howell.
- Variance of Standard is a warning sign: if real Standard performance is unpredictable, risk is higher than its mean suggests.

6. TASK 5 – SECURITY-ANALYSIS ENGINEERING AND REPRODUCIBILITY (8 MARKS)

Task 5.1 – Build a Reproducible Analysis Pipeline (2 marks)

The program in Appendix A is organised into labelled sections and functions, runs from beginning to end with no manual edits, and calls set.seed(55) (and set.seed for every simulation).

Stage	Function / section	Purpose
Data loading / simulation	load_hosts(), load_security_data(), read.csv of simulated files	Loads data, defines factors, stopifnot() checks size and missing values (input validation)
Cleaning and validation	clean_data()	Duplicates, impossible values, median imputation, log transforms
Descriptive analysis	sapply(...skewness, kurtosis), hist(), boxplot(), cor()	Distribution and relationships
Assumption checking	shapiro.test, bartlett.test, leveneTest, var.test, expected counts	Test selection evidence
Statistical testing	t.test, aov, chisq.test, fisher.test, oneway.test, kruskal.test, TukeyHSD	Hypothesis tests
Effect size	cohen.d, effect_eta2(), assocstats()/Cramér's V	Practical importance
Interpretation and reporting	choose_test(), analyse_defence(), report sections	Decisions and write-up

```
# 5.1 modular pipeline (functions are defined above: load_hosts, load_security_data,
clean_data, analyse_defence)
effect_eta2 <- function(fit) { ss <- summary(fit)[[1]]$"Sum Sq"; ss[1] / sum(ss) }
```

```
check_variance <- function(df) bartlett.test(blocked_rate ~ config, df)$p.value
choose_test <- function(df) if (check_variance(df) < 0.05) "Welch ANOVA / Kruskal-Wallis"
else "Classical ANOVA"
```

Task 5.2 – Validate the Statistical Implementation (2 marks)

Test case	**Expected behaviour (stated before execution)**	**Actual output**	**Match?**
TC1 – identical groups (same 30 values in 3 groups)	Between-group variation is zero: F = 0, p = 1 (not significant)	F = 0.0, p = 1.0	Yes
TC1b – three independent groups from the same distribution (N(70, 5))	Non-significant; p anywhere in (0,1), typically > 0.05	F = 0.45, p = 0.636	Yes
TC2 – assumption violation: Standard SD = 15 vs 5	Bartlett and Levene reject equal variance; choose_test() selects Welch/Kruskal	Bartlett p < 0.001; Welch p < 0.001	Yes
TC3 – boundary case: chi-square on 30 hosts (expected < 5)	R warning 'Chi-squared approximation may be incorrect'; Fisher/simulation used instead	Warning produced; 4/6 expected counts < 5; Fisher p = 0.0014	Yes
TC4 – boundary: empty cell (Segmented/Yes = 0	Expected counts still positive, test runs, standardised residual	Segmented/Yes observed = 0, expected = 3.0;	Yes

in the 30-host table)	not NaN	Monte-Carlo p = 0.0035	

Discrepancies explained. In TC1b the p-value is not exactly 1 because random samples from the same distribution still differ by chance; only identical samples (TC1) give F = 0. The Bartlett test did not flag the original 4-group data (p = 0.084) while Levene marginally did (p = 0.028); the two tests have different sensitivity to non-normality and group size, which is why Levene/Bartlett are interpreted together with boxplots and not alone.

Task 5.3 – Investigate a Potential Analytical Error (2 marks)

Deliberate error introduced: analysing paired measurements (30 hosts, latency in ms before and after a firewall-rule change) with an **unpaired** two-sample t-test – an inappropriate test selection that ignores the pairing.

```
lat <- read.csv('latency_paired.csv')
t.test(lat$before_ms, lat$after_ms)                # WRONG: treats samples as independent
t.test(lat$before_ms, lat$after_ms, paired = TRUE)   # CORRECT: same hosts measured twice
dz <- mean(lat$before_ms-lat$after_ms)/sd(lat$before_ms-lat$after_ms)
```

Incorrect versus corrected result

Analysis	t	df	p-value	Conclusion
WRONG – unpaired	1.57	58	0.122	Not significant: ‘rule change had no effect’ (WRONG)
CORRECT – paired	6.08	29	< 0.001	Significant: mean reduction 6.7 ms, 95% CI 4.4 to 8.9; Cohen's dz = 1.11 (large)

Source of the error. Host-to-host variation in baseline latency (SD ≈ 18 ms) is much larger than the within-host change (≈ 6.7 ms). The unpaired test puts that between-host variation into the denominator, hiding a consistent improvement – a Type II error. The paired test analyses the within-host differences, removing the between-host variation. **Change in interpretation:** the wrong analysis would have led to rejecting a beneficial rule change; the correct one shows a consistent, large latency reduction.

A second error demonstrated – ignoring multiple comparisons. Running all six uncorrected pairwise t-tests on the original defence data gives p-values of 2×10^{-6} to 0.55; the Enhanced–Premium p-value is 0.55 (not significant) in both uncorrected and Tukey analyses, but the uncorrected approach would also report borderline pairs as significant whenever a family-wise error occurs (26.5% chance under H0). Tukey HSD controls this (Task 4.2).

Task 5.4 – Final Security Report (2 marks)

Stage	What was done	Key result
Questions	Config–compromise association; protocol/attack; bytes, duration; defence comparison	Five security questions
EDA	Cleaned 1,512 → 1,499 records; heavy-tailed variables; ICMP attack concentration	Three non-obvious patterns
Predictions	Recorded before testing	3/3 supported
Tests	χ^2, goodness-of-fit, Fisher, Welch t, ANOVA, Tukey, Welch ANOVA, Kruskal–Wallis	Assumptions checked before each test
Recommendations	Harden Default hosts; rate-limit ICMP; do not use mean bytes as detector; deploy	Evidence-linked

	Enhanced (pilot first)	

Required distinctions

Distinction	In this study
Statistical evidence vs personal assumptions	The belief that Default hosts are riskier was supported by data ($\chi^2(2) = 44.0$, $V = 0.30$); the assumption that attacks send more bytes was not supported ($d = 0.09$).
Statistical vs practical significance	With n = 500–1,500 small effects become significant. Here effect sizes (V, η^2, d) show which results matter: large for protocol/duration and configuration, negligible for bytes.
Association vs causation	Configuration, protocol and ruleset were associated with outcomes; only the lab experiment (random assignment of equivalent hosts) supports a causal reading of ruleset effects, and it is simulated.
Observed results vs limitations	Stand-in/benchmark data; simulated defence data; non-independence of connections; unknown costs; labels may be imperfect.

What a decision-maker can reasonably conclude: harden Default-configured hosts first (largest, best-evidenced risk factor); treat ICMP traffic with rate-limiting and monitoring; do not rely on average traffic volume to detect attacks; deploy Enhanced rather than Premium or Standard if the cost of a breach exceeds the extra operating cost – after a pilot. **Further investigation needed:** repeat the EDA/tests on real NSL-KDD or live logs; collect real blocked-rate and false-positive data; identify cost per breach; account for repeated measures from the same attacker/host; study region and exposure effects with logistic regression or a Cochran–Mantel–Haenszel test (Exposure and Region were collected but not used).

7. Mandatory Submission Evidence and Mark-Wise Deliverable Map

Task	Marks	Deliverables supplied in this report	Section
1.1	2	Observed table with margins, H0/H1, df = 2, justification	2
1.2	2	Two (three) manual expected-count calculations, chisq.test() verification, statistic/df/p, interpretation	2
1.3	2	Standardised residuals, Cramér's V, mosaic plot and grouped bars	2
1.4	2	Goodness-of-fit with p = c(.5,.3,.2): hypotheses, statistic, p, conclusion	2
1.5	2	30-host table, expected counts, R warning, violated assumption, Fisher/simulation/merging, justification	2
2.1–2.5	10	Dataset description, variable types, cleaning, IQR/z outliers, descriptive stats, skewness/kurtosis, plots, correlations, contingency tables, 4 patterns, EDA summary	3
3 A–E	12	Three questions with	4

		H0/H1, predictions before testing, assumption checks, tests with CI and effect sizes, prediction comparison, defended conclusions	
4.1–4.3	10	FWER, simulation, boxplots, ANOVA, Tukey, η^2, assumption violation, Welch/Kruskal, cost-aware recommendation	5
5.1–5.4	8	Modular pipeline, ≥ 2 test cases with expected-vs-actual, deliberate error and correction, final report	6

Documents to submit together: this report (Word), Ethical_Hacking_4B_analysis.R, and the five CSV data files. Paste your own R console screenshots (with code visible) beside the shaded output blocks after running the script; do not submit screenshots that show only a p-value or graph.

APPENDIX A – COMPLETE R SOURCE CODE

Header and setup

```
#
================================================================================
=
# ITA14 ETHICAL HACKING - ASSIGNMENT 4B (CO4) : SECURITY STATISTICAL
ENGINEERING
# Run top-to-bottom. Put the CSV files in the working directory.
# Packages: vcd, moments, effsize, car   (install.packages(c("vcd","moments","effsize","car")))
#
================================================================================
=
suppressPackageStartupMessages({library(vcd); library(moments); library(effsize);
library(car)})
set.seed(55)
```

Task 1

```
load_hosts <- function(path = "hosts_dataset.csv") {
  d <- read.csv(path, stringsAsFactors = TRUE)
  d$Config      <- factor(d$Config,      levels = c("Default", "Hardened", "Segmented"))
  d$Exposure    <- factor(d$Exposure,    levels = c("Low", "Medium", "High"))
  d$Region      <- factor(d$Region,      levels = c("Urban", "Semi-urban", "Rural"))
  d$Compromised <- factor(d$Compromised, levels = c("Yes", "No"))
  stopifnot(nrow(d) >= 400, !anyNA(d))
  d
}
d <- load_hosts()
# 1.1
tab <- table(d$Config, d$Compromised); print(addmargins(tab))
```

```
# 1.2
rt <- rowSums(tab); ct <- colSums(tab); gt <- sum(tab)
E_manual_1 <- rt["Default"]   * ct["Yes"] / gt      # worked calculation 1
E_manual_2 <- rt["Hardened"]  * ct["No"]  / gt      # worked calculation 2
cat("E[Default,Yes] =", E_manual_1, "\nE[Hardened,No] =", E_manual_2, "\n")
ch <- chisq.test(tab); print(ch$expected); print(ch)
# 1.3
print(ch$stdres)
cramer_v <- sqrt(unname(ch$statistic) / (gt * (min(dim(tab)) - 1))); cat("Cramer's V =",
cramer_v, "\n")
print(assocstats(tab)$cramer)
mosaicplot(tab, shade = TRUE, main = "Configuration x Compromised")
barplot(t(tab), beside = TRUE, col = c("firebrick", "steelblue"), legend.text = TRUE,
        main = "Compromise by configuration", ylab = "Hosts")
# 1.4
obs <- table(d$Region[d$Compromised == "Yes"])
print(obs); print(chisq.test(obs, p = c(0.50, 0.30, 0.20)))   # order: Urban, Semi-urban, Rural
# 1.5
small <- d[sample(nrow(d), 30), ]
st <- table(small$Config, small$Compromised); print(st)
w  <- tryCatch(chisq.test(st), warning = function(w) { print(w);
suppressWarnings(chisq.test(st)) })
print(w$expected)
print(fisher.test(st))                                  # exact (Freeman-Halton) test
set.seed(55); print(chisq.test(st, simulate.p.value = TRUE, B = 2000))
merged <- table(ifelse(small$Config == "Default", "Default", "Other"), small$Compromised)
print(merged); print(fisher.test(merged))
```

Task 2

```
# Real dataset: NSL-KDD (Canadian Institute for Cybersecurity, KDDTrain+.txt) - use it when available.
# The CSV below is a KDD-style stand-in with the same variable names so the code runs end-to-end.
load_security_data <- function(path = "kdd_style_security_dataset.csv") read.csv(path, stringsAsFactors = FALSE)
raw <- load_security_data(); str(raw); print(summary(raw))
cat("Missing per column:\n"); print(colSums(is.na(raw))); cat("Duplicates:", sum(duplicated(raw)), "\n")
clean_data <- function(x) {
  x <- x[!duplicated(x), ]                    # remove exact duplicate records
  x <- x[x$duration >= 0, ]                   # negative duration is impossible
  x$dst_bytes[is.na(x$dst_bytes)]     <- median(x$dst_bytes, na.rm = TRUE)
  x$serror_rate[is.na(x$serror_rate)] <- median(x$serror_rate, na.rm = TRUE)
  x$protocol_type <- factor(x$protocol_type); x$class <- factor(x$class)
  x$log_src <- log10(x$src_bytes + 1); x$log_dur <- log1p(x$duration); x$log_dst <- log10(x$dst_bytes + 1)
  x
}
dc <- clean_data(raw); cat("Rows after cleaning:", nrow(dc), "\n")
num <- c("duration", "src_bytes", "dst_bytes", "count", "serror_rate")
iqr_out <- function(v) { q <- quantile(v, c(.25, .75)); i <- IQR(v); sum(v < q[1] - 1.5*i | v > q[2] + 1.5*i) }
print(sapply(dc[num], iqr_out)); print(sapply(dc[num], function(v) sum(abs(scale(v)) > 3)))
print(t(sapply(dc[num], function(v) c(mean = mean(v), median = median(v), sd = sd(v), IQR = IQR(v),
                                      skew = skewness(v), kurt = kurtosis(v)))))
par(mfrow = c(2, 4))
for (v in c("duration", "src_bytes", "count", "serror_rate")) hist(dc[[v]], main = paste("Histogram:", v), xlab = v, col = "steelblue")
```

```
for (v in c("duration", "src_bytes", "count", "serror_rate")) boxplot(dc[[v]], horizontal = TRUE,
main = paste("Boxplot:", v))
par(mfrow = c(1, 1))
print(round(cor(dc[num]), 3)); print(round(cor(dc[num], method = "spearman"), 3))
par(mfrow = c(2, 2))
plot(dc$log_dur, dc$log_src, col = ifelse(dc$class == "attack", "red", "blue"), pch = 20, xlab =
"log duration", ylab = "log src_bytes")
plot(dc$count, dc$serror_rate, col = ifelse(dc$class == "attack", "red", "blue"), pch = 20, xlab =
"count", ylab = "serror_rate")
boxplot(log_src ~ protocol_type, dc, main = "log src_bytes by protocol")
boxplot(count ~ class, dc, main = "count by class")
par(mfrow = c(1, 1))
ptab <- table(dc$protocol_type, dc$class); print(addmargins(ptab));
print(round(prop.table(ptab, 1), 3))
```

Task 3

```
# T-test
atk <- dc$log_src[dc$class == "attack"]; nrm <- dc$log_src[dc$class == "normal"]
print(shapiro.test(sample(atk, 500))); print(shapiro.test(sample(nrm, 500)))
qqnorm(atk); qqline(atk); qqnorm(nrm); qqline(nrm)
print(var.test(atk, nrm)); print(leveneTest(log_src ~ class, dc))
tt <- t.test(log_src ~ class, data = dc, var.equal = FALSE); print(tt)   # Welch
print(cohen.d(log_src ~ class, data = dc))
# ANOVA
a <- aov(log_dur ~ protocol_type, dc); print(summary(a))
print(shapiro.test(sample(resid(a), 500))); qqnorm(resid(a)); qqline(resid(a))
print(bartlett.test(log_dur ~ protocol_type, dc)); print(leveneTest(log_dur ~ protocol_type, dc))
print(oneway.test(log_dur ~ protocol_type, dc, var.equal = FALSE)); print(kruskal.test(log_dur
~ protocol_type, dc))
print(TukeyHSD(a)); ss <- summary(a)[[1]]$"Sum Sq"; cat("eta^2 =", ss[1] / sum(ss), "\n")
```

```
# Chi-square
ct3 <- table(dc$protocol_type, dc$class); ch3 <- chisq.test(ct3)
print(ch3$expected); print(ch3); print(ch3$stdres); print(assocstats(ct3)$cramer)
```

Task 4

```
k <- 4; m <- choose(k, 2); fwer <- 1 - 0.95^m; cat("pairs =", m, " FWER =", fwer, "\n")
sim <- read.csv("defence_simulated_original.csv", stringsAsFactors = TRUE)
sim$config <- factor(sim$config, levels = c("Basic", "Standard", "Enhanced", "Premium"))
# (R-native simulation, same design:)  set.seed(44); rate <- c(rnorm(30,64,5), rnorm(30,70,5),
rnorm(30,78,5), rnorm(30,79,5))
analyse_defence <- function(df) {
  print(aggregate(blocked_rate ~ config, df, function(x) c(n = length(x), mean = mean(x), sd =
sd(x), median = median(x))))
  boxplot(blocked_rate ~ config, df, xlab = "Configuration", ylab = "Blocked-attack rate (%)",
col = "lightblue")
  fit <- aov(blocked_rate ~ config, df); print(summary(fit))
  print(shapiro.test(resid(fit))); qqnorm(resid(fit)); qqline(resid(fit))
  print(bartlett.test(blocked_rate ~ config, df)); print(leveneTest(blocked_rate ~ config, df))
  print(TukeyHSD(fit)); ss <- summary(fit)[[1]]$"Sum Sq"; cat("eta^2 =", ss[1] / sum(ss), "\n")
  print(oneway.test(blocked_rate ~ config, df, var.equal = FALSE));
print(kruskal.test(blocked_rate ~ config, df))
  invisible(fit)
}
fit_orig <- analyse_defence(sim)
sim2 <- read.csv("defence_simulated_inflated.csv", stringsAsFactors = TRUE)   # Standard sd
inflated to 15
sim2$config <- factor(sim2$config, levels = levels(sim$config))
fit_infl <- analyse_defence(sim2)
```

Task 5

```
# 5.1 modular pipeline (functions are defined above: load_hosts, load_security_data,
clean_data, analyse_defence)
effect_eta2 <- function(fit) { ss <- summary(fit)[[1]]$"Sum Sq"; ss[1] / sum(ss) }
check_variance <- function(df) bartlett.test(blocked_rate ~ config, df)$p.value
choose_test <- function(df) if (check_variance(df) < 0.05) "Welch ANOVA / Kruskal-Wallis"
else "Classical ANOVA"
# 5.2 test cases
identical_groups <- data.frame(config = factor(rep(c("A","B","C"), each = 30)),
                               y = rep(rnorm(30, 70, 5), 3))
cat("TC1 expected p = 1:\n"); print(summary(aov(y ~ config, identical_groups)))
cat("TC2 expected: variance violation detected ->", choose_test(sim2), "\n")
cat("TC3 expected: small-sample warning:\n"); print(tryCatch(chisq.test(st), warning =
function(w) conditionMessage(w)))
# 5.3 deliberate error: unpaired test on paired data
lat <- read.csv("latency_paired.csv")
cat("WRONG  (unpaired):\n"); print(t.test(lat$before_ms, lat$after_ms))
cat("RIGHT  (paired):\n");   print(t.test(lat$before_ms, lat$after_ms, paired = TRUE))
dz <- mean(lat$before_ms - lat$after_ms) / sd(lat$before_ms - lat$after_ms); cat("Cohen's dz
=", dz, "\n")
cat("Uncorrected pairwise p-values (wrong) vs Tukey (right):\n")
print(pairwise.t.test(sim$blocked_rate, sim$config, p.adjust.method = "none"));
print(TukeyHSD(aov(blocked_rate ~ config, sim)))
sessionInfo()
```